

make

代码变成可执行文件，叫做编译(compile)；先编译这个，还是先编译那个，叫做构建(build)。

make是最常用的构建工具，诞生于1977年，主要用于C语言的项目。

1.make的概念

make意为制作。make命令就是要做出某个文件。比如要做出文件a.txt，就可以执行：

```
1 | $ make a.txt
```

但是如果你输入这条命令，它并不会起作用。因为make命令本身并不知道，如何做出a.txt，需要有人告诉它，如何调用其他命令完成这个目标。

比如，假设文件a.txt依赖于b.txt和c.txt，是后面两个文件连接的产物。那么，make需要知道下面的规则：

```
1 | a.txt:b.txt c.txt
2 |     cat b.txt c.txt > a.txt
```

也就是说，make a.txt这条命令的背后，实际上分成两步：第一步，确认b.txt和c.txt已经存在，第二步使用cat命令将这两个文件合并，输出为新文件。

像这样的规则，都写在makefile文件中，make命令依赖于这个文件进行构建。

makefile文件可以用命令行参数指定为其他文件名。

```
1 | $ make -f rules.txt
2 | # 或者
3 | $ make --file=rules.txt
```

上面代码指定make命令依据rules.txt文件中的规则进行构建。

总之，make只是一个根据指定的shell命令进行构建的工具。它的规则很简单，你规定要构建那个文件、它依赖哪些文件，当那些文件有变动时，如何重新构建它。

2.Makefile文件的格式

2.1 概述

Makefile文件由一系列规则构成。每条规则的形式如下：

```
1 | <target> : <prerequisites>
2 | [tab] <commands>
```

上面第一行冒号前面的部分，叫做目标，冒号后面的部分叫做前置条件；第二行必须由一个tab键起首，后面跟着命令。

目标是必须的，不可省略；前置条件和命令都是可选的，但两者之中必须至少存在一个。

每条规则明确两件事：构建目标的前置条件是什么，以及如何构建。

2.2 目标

一个目标就构成一条规则。目标通常是文件名，指明make命令所要构建的对象，比如上文的a.txt。目标可以是一个文件名，也可以是多个文件名，之间用空格分隔。

除了文件名，目标还可以是某个操作的名字，这称为“伪目标”。

```
1 clean:
2     rm *.o
```

上面代码的目标是clean，它不是文件名，而是一个操作的名字，作用是删除文件对象。

```
1 $ make clean
```

但是，如果当前目录中，正好有一个文件叫clean那么这个命令不会执行。因为make发现clean文件已经存在，就认为没有必要重新构建了，就不会执行指定的rm命令。

为了避免这种情况，可以明确声明clean是伪目标：

```
1 .PHONY:clean
2 clean:
3     rm *.o temp
```

声明clean是伪目标之后，make就不会去检查是否存在一个叫做clean的文件，而是每次运行都执行的命令。

如果make命令运行时没有指定目标，默认执行makefile文件的第一个目标。

2.3 前置条件

前置条件通常是一组文件名，之间用空格分隔。它指定了目标是否重新构建的判断标准：

只要有一个前置文件不存在，或者有过更新，目标就需要重新构建。

```
1 result.txt:source.txt
2     cp source.txt result.txt
```

上述代码中，构建result.txt的前置条件是source.txt。如果当前目录中，source.txt已经存在，那么make result.txt可以正常运行，否则必须再写一条规则，来生成source.txt。

```
1 source.txt:
2     echo "this is the source" > source.txt
```

上述代码中，source.txt后面没有前置条件，就意味着它跟其他文件都无关，只要这个文件还不存在，每次调用make source.txt，它都会生成。

如果要生成多个文件，往往采用下面的写法：

```
1 source:file1 file2 file3
```

上面代码中，source是一个伪目标，只有三个前置条件，没有任何对应的命令。

```
1 | $ make source
```

执行上述命令后，就会一次性生成file1,file2,file3三个文件。

2.4 命令

命令表示如何更新目标文件，由一行或多行的shell命令组成。它是构建目标的具体指令，它的运行结果通常就是生成目标文件。

每行命令之前必须有一个tab键。如果想用其他键，可以用内置变量.RECIPEPREFIX声明。

```
1 | .RECIPEPREFIX = >
2 | all:
3 | > echo Hello,world
```

上面代码用.RECIPEPREFIX指定，大于号替代tab键。所以，每一行命令的起首变成了大于号。

需要注意的是，每行命令在一个单独的shell中执行。这些shell之间没有继承关系。

```
1 | var-lost:
2 |     export foo=bar
3 |     echo "foo=[$$foo]"
```

上面代码执行make var-lost后，取不到foo的值。因为两行命令在两个不同的进程执行。一个解决方法是将两行命令写在一行，中间用分号分隔。

```
1 | var-kept:
2 |     export foo=bar;echo "foo=[$$foo]"
```

另一个方法是在换行符前加反斜杠转义。

```
1 | var-kept:
2 |     export foo=bar; \
3 |     echo "foo=[$$foo]"
```

最后一个方法是加上.ONESHELL:命令。

```
1 | .ONESHELL:
2 | var-kept:
3 |     export foo=bar;
4 |     echo "foo=[$$foo]"
```

3.Makefile文件的语法

3.1 注释

#表示注释。

```
1 # 这是注释
2 result.txt: source.txt
3     # 这是注释
4     cp source.txt result.txt # 这也是注释
```

3.2 回声

正常情况下，make会打印每条命令，然后再执行，这就叫做回声（echoing）。

```
1 test:
2     # 这是测试
```

执行上面的规则，会得到下面的结果：

```
1 $ make test
2 # 这是测试
```

在命令的前面加上@，就可以关闭回声。

```
1 test:
2     @# 这是测试
```

现在再执行make test，就不会有任何输出。

由于在构建过程中，需要了解当前在执行哪条命令，所以通常只在注释和纯显示的echo命令前面加上@。

```
1 test:
2     @# 这是测试
3     @echo TODO
```

3.3 通配符

通配符（wildcard）用来指定一组符合条件的文件名。Makefile的通配符于Bash一致，主要有星号（*）、问号（?）和[...]。比如，*.o表示所有后缀名为o的文件。

```
1 clean:
2     rm -f *.o
```

3.4 模式匹配

make命令允许对文件名，进行类似正则运算的匹配，主要用到的匹配符是%。比如，假定当前目录下有f1.c和f2.c两个源码文件，需要将他们编译为对应的对象文件。

```
1 %.o:%.c
```

等同于下面的写法：

```
1 f1.o:f1.c
2 f2.o:f2.c
```

使用匹配符%，可以将大量同类型的文件，只用一条规则就完成构建。

3.5 变量和赋值符

Makefile允许使用等号自定义变量。

```
1 txt=Hello world
2 test:
3     @echo $(txt)
```

上面代码中，变量txt等于Hello World。调用时，变量需要放在\$()中。

调用shell变量，需要在美元符号前再加一个美元符号，这是因为make命令会对美元符号转义。

```
1 test:
2     @echo $$HOME
```

有时，变量的值可能指向另一个变量。

```
1 v1=$(v2)
```

上面代码中，变量v1的值是另一个变量v2。这时会产生一个问题，v1的值到底在定义时扩展（静态扩展），还是在运行时扩展（动态扩展）？

Makefile提供了四个赋值运算符（=、:=、? =、+=），

```
1 VARIABLE = value
2 # 在执行时扩展，允许递归扩展。
3
4 VARIABLE := value
5 # 在定义时扩展。
6
7 VARIABLE ?= value
8 # 只有在该变量为空时才设置值。
9
10 VARIABLE += value
11 # 将值追加到变量的尾端。
```

3.6 内置变量

make命令提供一系列内置变量，比如，\$(CC)指向当前使用的编译器，\$(MAKE)指向当前使用的Make工具。这主要是为了跨平台的兼容性。

```
1 output:
2     $(CC) -o output input.c
```

3.7 自动变量

Make命令还提供一些自动变量，他们的值与当前规则有关。

1. \$@

\$@指代当前目标，就是make命令当前构建的那个目标。比如，make foo的\$@就指代foo

```
1 | a.txt b.txt:
2 |     touch $@
```

等同于下面:

```
1 | a.txt:
2 |     touch a.txt
3 | b.txt:
4 |     touch b.txt
```

2. \$<

\$< 指代第一个前置条件。比如, 规则为t: p1 p2,那么\$<就指代p1。

```
1 | a.txt:b.txt c.txt
2 |     cp $< $@
```

等同于下面:

```
1 | a.txt:b.txt c.txt
2 |     cp b.txt a.txt
```

3. \$?

\$?指代比目标更新的所有前置条件, 之间以空格分隔。比如, 规则为t: p1 p2,其中p2的时间戳比t新, \$?就指代p2。

4. \$^

指代所有前置条件, 之间以空格分隔。比如, 规则为 t: p1 p2, 那么 \$^ 就指代 p1 p2 。

5. \$*

指代匹配符 % 匹配的部分, 比如% 匹配 f1.txt 中的f1, \$* 就表示 f1。

6. \$(@D) 和 \$(@F)

\$(@D) 和 \$(@F) 分别指向 \$@ 的目录名和文件名。比如, \$@是 src/input.c, 那么\$(@D) 的值为 src, \$(@F) 的值为 input.c。

3.8 判断和循环

Makefile使用Bash语法, 完成判断和循环。

```
1 | ifeq ($(CC),gcc)
2 |     libs=$(libs_for_gcc)
3 | else
4 |     libs=$(normal_libs)
5 | endif
```

上面代码判断当前编译器是否gcc, 然后指定不同的库文件。

```

1 LIST = one two three
2 all:
3     for i in $(LIST); do \
4         echo $$i; \
5     done
6
7 # 等同于
8
9 all:
10    for i in one two three; do \
11        echo $i; \
12    done

```

3.9 函数

Makefile还可以使用函数：

```

1 $(function arguments)
2 # 或者
3 ${function arguments}

```

Makefile提供了许多内置函数：

- shell函数：执行shell命令

```
1 srcfiles := $(shell echo src/{00..99}.txt)
```

- wildcard函数：替换Bash的通配符

```
1 srcfiles := $(wildcard src/*.txt)
```

- subst函数：用来文本替换

```
1 $(subst from,to,text)
```

下面的例子将字符串“feet on the street”替换成“fEEt on the strEEt”。

```
1 $(subst ee,EE,feet on the street)
```

- patsubst函数：用于模式匹配的替换

```
1 $(patsubst pattern,replacement,text)
```

下面的例子将文件名“x.c.c bar.c”替换成“x.c.o bar.o”。

```
1 $(patsubst %.c,%.o,x.c.c bar.c)
```

- 替换后缀名：变量名+冒号+后缀名替换规则。

```
1 min:$(OUTPUT:.js=.min.js)
```

上面代码的意思是，将变量OUTPUT中的后缀名为.js全部替换成.min.js。

4.Makefile的实例

4.1 执行多个目标

```
1  .PHONY: cleanall cleanobj cleandiff
2
3  cleanall : cleanobj cleandiff
4          rm program
5
6  cleanobj :
7          rm *.o
8
9  cleandiff :
10         rm *.diff
```

上面的代码可以调用不同的目标，删除不同后缀名的文件，也可以调用一个目标，删除所有指定类型的文件。

4.2 编译C语言项目

```
1  edit : main.o kbd.o command.o display.o
2      cc -o edit main.o kbd.o command.o display.o
3
4  main.o : main.c defs.h
5      cc -c main.c
6  kbd.o : kbd.c defs.h command.h
7      cc -c kbd.c
8  command.o : command.c defs.h command.h
9      cc -c command.c
10 display.o : display.c defs.h
11      cc -c display.c
12
13 clean :
14      rm edit main.o kbd.o command.o display.o
15
16 .PHONY: edit clean
```